

Voxter: A Real-Time Visuomotor Agent for Generalized Geometry Dash Play

Model Theory and Implementation Contracts

Voxter Project

May 25, 2026

Abstract

Voxter is proposed as a real-time visuomotor policy agent for Geometry Dash. Its purpose is not to reproduce memorized click timestamps for a fixed level, but to learn transferable gameplay behavior: reading visual structure, predicting near-future hazards, and producing the correct binary input state in real time. The central claim of the project is that a model trained on a randomized gameplay generator can acquire reusable control behavior that generalizes to unseen standard levels better than a timestamp bot.

The project begins from human demonstrations collected on a randomized training level. The training level is an endless sequence of randomized sections derived from the main RobTop levels. At each new section, properties such as player form, cosmetic icon appearance, gameplay speed, Ghost Mode fading, and randomized path choices may change. This makes the level a domain-randomized source of gameplay situations rather than a fixed target level. Voxter therefore treats the training level as a generator of scenes, trajectories, and control problems.

The learning pipeline has three stages. Stage 1 trains a reactive behavioral cloning policy from preprocessed visual observations to binary input states. Stage 2 extends the policy with memory, preferably beginning with a GRU, so that the agent can infer velocity, mode context, previous action effects, and other partially observable quantities. Stage 3 fine-tunes the imitation-initialized policy with reinforcement learning under the constraint that the agent must play in real time exactly like a human, without pausing, slowing down, or using privileged future information.

The document formalizes the observation space, action space, causal alignment principle, dataset construction rules, model architecture, learning objectives, reinforcement learning objective, evaluation protocol, generalization criteria, implementation contracts, and expected failure modes.

Contents

1	Foundations and Thesis	3
1.1	Motivation: From Timestamp Bots to Gameplay Agents	3
1.2	Core Thesis of Voxter	3
1.3	The Geometry Dash Control Problem	4
2	Training Generator and Generalization	5
2.1	Training Level as a Randomized Gameplay Generator	5
2.2	Generalization Objective	6
3	Observation and Action Representation	7
3.1	Observation Space	7
3.2	Icon Standardization	7
3.3	Binary Action Space	8
3.4	Rejection of Timestamp Learning	8

4	Dataset Construction and Causal Contracts	9
4.1	Causal Alignment Principle	9
4.2	Capture and Dataset Construction Contracts	10
4.3	Successful and Failed Demonstrations	11
4.4	Repeated Attempts and Demonstration Diversity	11
4.5	Balancing Strategy	12
4.6	Ghost Mode as Visual Domain Randomization	12
5	Model Stages and Learning Objectives	12
5.1	Stage 1: Reactive Behavioral Cloning	12
5.2	Why Stage 1 Is Not Enough	13
5.3	Stage 2: Memory-Based Sequential Policy	14
5.4	Sequential Behavioral Cloning Objective	14
5.5	Exposure Bias and Compounding Error	15
5.6	Stage 3: Reinforcement Fine-Tuning	15
6	Evaluation, Deployment, and Runtime Contracts	16
6.1	Real-Time Deployment Contract	16
6.2	Offline Evaluation Protocol	17
6.3	Online Evaluation Protocol	18
6.4	Baselines	19
6.5	Model Selection and Hyperparameters	19
6.6	Data Splitting Rules	20
6.7	Partial Observability and Memory Justification	20
7	Implementation Interface, Risks, and Plan	21
7.1	Implementation Architecture	21
7.2	Reinforcement Learning Environment Interface	21
7.3	Privileged Information Rule	22
7.4	Failure Modes and Limitations	22
7.5	Ethical and Project-Scope Note	24
7.6	First-Version Development Plan	24
7.7	Minimal Dataset Schema	24
7.8	Open Questions for Later Iteration	25
7.9	Conclusion	25
A	Notation Table	26
B	Compact Mathematical Summary	26
C	Project Source Notes	27

1 Foundations and Thesis

1.1 Motivation: From Timestamp Bots to Gameplay Agents

A traditional Geometry Dash bot can be understood as a replay mechanism. It stores a sequence of input events at specific timestamps or positions and executes them later. This can work very well on a known level, but it is fundamentally brittle. The bot is coupled to the exact level, the exact speed profile, the exact starting conditions, and the exact synchronization assumptions under which the clicks were recorded.

This project aims at a different object. Voxter should not answer the question:

At what timestamp should the next click occur?

It should answer:

Given what is visible now, and given the recent history, should the input be held or released?

This distinction is the philosophical and technical center of the project. Timestamp learning is an attempt to memorize a script. Visual control is an attempt to learn a policy. A timestamp bot is level-specific. A visuomotor agent can, in principle, transfer to new levels because it conditions its behavior on observations rather than on absolute time.

Geometry Dash is a good environment for this idea because the game is simple at the action level but rich at the perception-control level. The input can be represented as binary: held or released. However, the meaning of that binary input depends on the current game mode, gravity, speed, momentum, portals, jump pads, orbs, obstacle geometry, and recent trajectory. The problem is therefore not action-space complexity. The difficulty lies in learning the correct mapping from visual structure and temporal context to precise control.

1.2 Core Thesis of Voxter

The core thesis is:

Voxter is not a Geometry Dash click predictor. Voxter is a real-time visuomotor policy agent trained from human demonstrations and reinforcement feedback to map visual observations and recent temporal context to binary input states. Its target behavior is transferable gameplay competence on unseen standard levels.

In this document, the terms are used as follows:

- **Agent** refers to the full deployed system: perception, policy, memory, input execution, and runtime constraints.
- **Policy** refers to the mathematical function that maps observations or observation histories to actions.
- **Controller** refers to the subsystem that converts the policy output into real-time input states.
- **Player** is used only informally when comparing Voxter to human Geometry Dash play.

The formal name of the model class is therefore:

an imitation-initialized, reinforcement-tuned visuomotor policy agent for a partially observable real-time game environment.

A shorter working description is:

a real-time visuomotor policy agent.

1.3 The Geometry Dash Control Problem

Geometry Dash is a deterministic or nearly deterministic side-scrolling game from the perspective of a fixed level and fixed starting conditions. However, the agent does not receive a symbolic representation of the level. It receives pixels. It must act from visual input under real-time constraints.

Let the environment have an unobserved state

$$s_t \in \mathcal{S}.$$

This state includes the player position, velocity, gravity, current mode, speed, level geometry, active portals, animation state, collision state, and other variables. The agent does not directly observe s_t . Instead, the game produces a captured frame

$$I_t = \Omega(s_t),$$

where Ω is the rendering and capture process. The model receives a preprocessed observation

$$o_t = P_{\text{pre}}(I_t).$$

The agent outputs a binary input state

$$a_t \in \mathcal{A} = \{0, 1\}.$$

The environment evolves according to

$$s_{t+1} = F(s_t, a_t),$$

where F is the game transition function. The model does not need to know F explicitly, but it must learn a policy that chooses useful actions under this transition system.

1.3.1 Timing Convention and Control Interval

Unless otherwise stated, a_t denotes the input state to be applied during the control interval $[t, t + 1)$ after the observation o_t has been made available to the policy. Under this convention, the policy observes o_t , chooses a_t , and the environment transition $s_t \rightarrow s_{t+1}$ is evaluated under that input state.

Raw logs may record the physical held state at capture time. That logged state may correspond to input that was already being applied before the captured frame was rendered. Dataset construction must therefore align logged input states so that the supervised label represents the action that was causally available from the corresponding observation history.

Because the observation is visual and incomplete, the problem is naturally partially observable. The optimal action may depend not only on o_t , but also on the observation-action history

$$\mathcal{H}_t = (o_1, a_1, \dots, o_{t-1}, a_{t-1}, o_t).$$

Thus the ideal policy has the form

$$\pi^*(a_t \mid \mathcal{H}_t).$$

A purely reactive policy approximates

$$\pi(a_t \mid o_t),$$

or, with a short frame stack,

$$\pi(a_t \mid o_{t-k+1:t}).$$

A memory-based policy approximates

$$\pi(a_t \mid o_{\leq t}, a_{< t}).$$

2 Training Generator and Generalization

2.1 Training Level as a Randomized Gameplay Generator

The training level is not treated as the final task. It is treated as a generator of gameplay situations.

Let the randomized training generator be G , and let P_G be the distribution over section configurations induced by G . At each section boundary, the generator samples a new section configuration

$$z_i \sim P_G.$$

A section configuration can be represented as

$$z_i = (g_i, m_i, c_i, \nu_i, \rho_i, \gamma_i),$$

where:

- g_i is the underlying geometry or obstacle pattern,
- m_i is the gameplay mode or player form, such as cube, ship, wave, ball, UFO, robot, spider, or swing,
- c_i is the cosmetic icon appearance when randomized separately,
- ν_i is the gameplay speed,
- ρ_i is the random path configuration,
- γ_i is the visual condition induced by Ghost Mode.

The generator creates a trajectory

$$\tau = ((o_1, a_1), (o_2, a_2), \dots, (o_T, a_T)).$$

During data collection, this trajectory is produced by a human demonstrator policy:

$$a_t \sim \pi_H(\cdot \mid \mathcal{H}_t).$$

The agent is then trained to approximate and eventually improve upon the useful structure of π_H .

The generator has two purposes:

1. Diversity: expose the model to many modes, speeds, paths, and visual conditions.
2. Anti-memorization pressure: make absolute timestamp replay ineffective as a general solution.

The training level therefore implements a form of domain randomization. Let P_{G,π_H} be the trajectory distribution induced by the randomized generator and the human demonstrator policy. Instead of training only on one fixed level, Voxter is trained over a distribution of gameplay situations:

$$\tau \sim P_{G,\pi_H}.$$

The target is not to master G as a single level. The target is to learn a control principle that transfers beyond G .

2.2 Generalization Objective

The project defines generalization in a practical and gameplay-centered way:

Voxter generalizes if, when deployed as a fixed trained policy on a brand new standard level, it can play through repeated attempts and eventually complete the level more effectively than a timestamp bot that has no level-specific script.

Online adaptation or level-specific fine-tuning may be studied separately, but it is not part of the main real-time transfer claim.

2.2.1 Fixed-Policy Attempt Semantics

In the main fixed-policy transfer setting, repeated attempts do not imply level-specific learning. They mean that the same trained policy is evaluated over multiple independent or reset attempts. Success is therefore measured by completion probability, best progress, mean attempts to completion, and robustness under real-time execution. A separate setting may later allow online adaptation across attempts.

There are two levels of generalization.

2.2.2 In-Generator Generalization

In-generator generalization refers to performance on unseen sections sampled from the same style of training generator. In this project, some sections are removed during training and added back during testing. These held-out sections are brand new sections added to the section set, but they are subject to the same randomization conditions as the training sections.

This evaluates whether Voxter learned beyond memorizing the exact training sections.

2.2.3 Out-of-Generator Transfer

Out-of-generator transfer refers to performance on standard levels outside the randomized generator. This is the ambitious final goal:

$$\text{train on randomized generated gameplay} \implies \text{play unseen standard levels.}$$

This is the stronger and more important claim. It asks whether the model has learned the visual grammar of Geometry Dash: spikes, platforms, orbs, portals, ship corridors, wave patterns, jump timing, and other reusable structures.

3 Observation and Action Representation

3.1 Observation Space

The raw observation source is the full captured game frame at the base screen resolution. Tensor shapes are written in height-width-channel order:

$$I_t \in \mathbb{R}^{H_0 \times W_0 \times 3}, \quad H_0 = 1080, \quad W_0 = 1920.$$

The raw frame is not directly used by the model. It is transformed into a preprocessed observation

$$o_t = P_{\text{pre}}(I_t),$$

where P_{pre} is a preprocessing function.

The first intended preprocessing pipeline is:

1. capture the full screen gameplay frame,
2. optionally crop only if stable non-game UI elements must be removed,
3. resize the frame to a fixed model resolution,
4. convert to grayscale,
5. normalize pixel values,
6. standardize cosmetic player-icon appearance if a reliable method exists,
7. construct frame stacks for short-horizon motion inference.

The model input for Stage 1 is therefore

$$x_t = (o_{t-k+1}, o_{t-k+2}, \dots, o_t),$$

where k is the number of stacked frames. Equivalently,

$$x_t \in \mathbb{R}^{H \times W \times k}.$$

The value of k should be treated as a tunable hyperparameter. A single frame may be insufficient because velocity, gravity, and momentum are not always inferable from static pixels. A short stack provides minimal temporal information while keeping Stage 1 simpler than a recurrent policy.

The model should infer game mode visually rather than receiving it as an explicit auxiliary input. This enforces the desired property that Voxter plays from the screen like a human.

3.2 Icon Standardization

The training generator may randomize both player form and cosmetic icon appearance. Form changes, such as cube, ship, or wave, are gameplay-dynamic conditions represented by m_i . Cosmetic appearance changes are represented by c_i . In principle, both kinds of variation improve robustness, but only cosmetic appearance is a candidate for standardization. The important variables are position, motion, mode, gravity, and collision context.

Therefore, icon standardization is allowed and recommended if it can be implemented reliably. The purpose is to reduce irrelevant appearance variation while preserving the gameplay-relevant state.

Let C be an icon canonicalization function. Then preprocessing may be represented as

$$o_t = P_{\text{pre}}(C(I_t)).$$

The canonicalization process must not introduce future information, privileged game-state access unavailable during deployment, or manually encoded labels. If implemented through computer vision, it should operate only on the captured frame at time t .

Ablation should eventually compare:

1. no icon standardization,
2. icon standardization,
3. randomized icon training without standardization.

The expected benefit is that standardization reduces sample complexity and prevents the model from wasting capacity on cosmetic variation.

3.3 Binary Action Space

The action space is the binary input state:

$$\mathcal{A} = \{0, 1\}.$$

The semantics are:

$$a_t = 0 \quad \text{input released,} \quad a_t = 1 \quad \text{input held.}$$

This is not the same as a click-event label. A click is a transition from released to held:

$$e_t^{\text{press}} = \mathbf{1}\{a_{t-1} = 0 \wedge a_t = 1\}.$$

A release is a transition from held to released:

$$e_t^{\text{release}} = \mathbf{1}\{a_{t-1} = 1 \wedge a_t = 0\}.$$

The held-state formulation is essential because Geometry Dash modes often require sustained input. Ship, wave, robot, swing, and related modes are not controlled only by isolated click moments. They depend continuously on whether the input is currently held.

Thus the dataset should store a_t , not merely e_t^{press} .

3.4 Rejection of Timestamp Learning

Voxter explicitly rejects timestamp learning as the primary objective.

A timestamp policy has the approximate form

$$a_t = f(t),$$

or, for a fixed level,

$$a_t = f(\ell_t),$$

where ℓ_t may be a deterministic progress coordinate through the known level. This can be effective for replay, but it does not produce transferable gameplay competence.

A visual-control policy has the form

$$a_t \sim \pi(a_t \mid \mathcal{H}_t),$$

where $\mathcal{H}_t$ is built from observations and previous actions. Such a policy can, in principle, act on new levels because the decision is conditioned on visible structure rather than on a memorized script.

The training generator is specifically designed to make timestamp memorization weak. Random section ordering, random paths, randomized speeds, icon variation, and Ghost Mode perturbations make any absolute time-based strategy brittle. Voxter must instead learn visual affordances:

- spikes imply jump avoidance,
- platforms imply landing opportunities,
- portals imply mode or gravity transitions,
- orbs imply timed activation opportunities,
- ship corridors imply continuous vertical control,
- wave patterns imply alternating held and released intervals,
- speed changes imply recalibrated timing.

4 Dataset Construction and Causal Contracts

4.1 Causal Alignment Principle

The causal alignment principle is one of the central implementation contracts of the project.

A training label assigned to an observation must correspond to an action chosen from information available at or before that observation, never from future visual consequences.

Naively pairing a captured frame o_t with an input state a_t may be wrong if the frame has already been affected by the action or if capture and input logs are misaligned.

Let $a_t^{\log}$ denote the raw logged held-state at capture index t . After system-delay correction, the supervised label for observation o_t is

$$y_t = a_{t+\delta_{\text{sys}}}^{\log}.$$

Here a positive δ_{sys} means that the action log lags behind the visual frame by δ_{sys} ticks, so the label must be taken from a later log index. After this correction, the aligned supervised target is treated as the document's a_t :

$$a_t \equiv y_t, \quad y_t \sim \pi_H(\cdot \mid \mathcal{H}_t).$$

This sign convention is part of the dataset contract. If an implementation uses the opposite convention, it must translate its offset before reporting δ_{sys} .

Samples for which $t + \delta_{\text{sys}}$ falls outside the valid episode range should be discarded or padded according to the dataset contract.

There are at least two delays to distinguish:

δ_{system} : capture, logging, rendering, and input synchronization delay,

δ_{human} : human perception, decision, and motor delay.

The system delay should be measured and corrected. The human delay should not be naively removed, because expert gameplay contains anticipatory decisions. A skilled player may press before an obstacle not because of delayed reaction, but because the visual structure predicts the future need for input.

The recommended alignment procedure is:

1. record frames and input state with timestamps or game-tick indices,
2. measure system-level delay through a calibration script,
3. shift labels or frames to correct technical misalignment,
4. preserve expert anticipatory behavior,
5. tune a small residual offset only if validation shows systematic early or late actions,
6. verify alignment by online playback performance, not only offline accuracy.

4.2 Capture and Dataset Construction Contracts

The custom capture script must produce enough information for supervised learning, temporal alignment, debugging, and real-time deployment validation.

At minimum, each captured sample should contain

$$(I_t, a_t, \kappa_t),$$

where:

- I_t is the raw captured frame,
- a_t is the input held-state,
- κ_t is the timestamp or game-tick index.

If possible, the capture script should also record metadata:

$$q_t = (\text{alive}_t, \text{done}_t, \ell_t, m_t, c_t, \nu_t, \text{seed}_t, \text{section}_t),$$

where:

- alive_t indicates whether the attempt is still active,
- done_t indicates death, completion, or another terminal reset condition,
- ℓ_t indicates completion or progress,
- m_t is game mode if accessible,

- c_t is icon appearance or cosmetic metadata if accessible,
- ν_t is speed if accessible,
- seed_t is the random seed if available,
- section_t identifies the current generated section if available.

This metadata is not necessarily passed to the model. It is used for dataset analysis, balancing, ablation, and evaluation.

The ideal sampling rate is the game-tick rate. If game-tick capture is too expensive or unstable, a fixed sampling rate may be used, but then the sampling rate becomes part of the formal deployment contract. Training and deployment should use compatible temporal assumptions.

4.3 Successful and Failed Demonstrations

The dataset should primarily contain useful gameplay behavior. However, failed attempts can still contain useful prefixes. A failed run often contains many correct decisions before the fatal mistake.

The recommended rule is:

Keep playable demonstration prefixes; discard death aftermath and clearly invalid control segments.

Let T_d be the death time of an attempt. Instead of discarding the whole attempt, one may keep:

$$\tau_{\text{valid}} = ((o_1, a_1), \dots, (o_{T_d-\epsilon}, a_{T_d-\epsilon})),$$

where ϵ removes the immediate unreliable segment before death if needed.

The value of ϵ is a hyperparameter. If the fatal mistake is visible only very close to death, most of the trajectory can be useful. If the trajectory enters an unrecoverable state long before death, a larger prefix truncation is required.

Death animations, post-death frames, menu transitions, attempt reset screens, and idle periods should not be used as normal gameplay training data.

4.4 Repeated Attempts and Demonstration Diversity

Repeated attempts of the same section should be included. Even when the same section appears, the human action pattern is not exactly identical. Small variations in timing, held durations, and recovery behavior provide useful diversity.

However, repeated attempts can also cause overfitting if the model sees the same exact obstacle sequence too many times. The dataset should therefore track section frequency when possible.

Let n_j be the number of samples or trajectories from section j . A simple section-aware sampling rule is:

$$\Pr(J = j) \propto \min(n_j, n_{\max}),$$

or alternatively sample sections approximately uniformly when section IDs are available.

If section IDs are unavailable, approximate balancing can be performed by mode, speed, death location, or visual clustering.

4.5 Balancing Strategy

The binary action distribution may be imbalanced. For example, many frames may be released, especially in cube sections. A naive model may learn a hard bias toward doing nothing.

However, forcing a perfectly balanced dataset can distort the true control distribution. The correct goal is not to make the agent hold and release equally often. The goal is to make the agent sensitive to when holding is appropriate.

The recommended objective is weighted binary cross-entropy:

$$\mathcal{L}_{\text{BC}}(\theta) = - \sum_t [w_1 a_t \log p_t + w_0 (1 - a_t) \log (1 - p_t)],$$

where

$$p_t = \pi_\theta(a_t = 1 \mid x_t).$$

The weights w_0 and w_1 may be computed from class frequencies, tuned manually, or adapted per game mode.

Game-mode balancing is also important. Cube, ship, wave, ball, UFO, robot, spider, and swing-like behaviors have different action distributions and different consequences of holding. If explicit mode labels are available as metadata, they should be used for dataset auditing and sampler design, even if they are not given to the model at inference time.

Speed and obstacle-density balancing are harder to control realistically and should not be required in the first version. They can still be measured for analysis.

4.6 Ghost Mode as Visual Domain Randomization

Ghost Mode fades objects in, out, or both. In the learning theory of Voxter, Ghost Mode is best described as environment-level visual domain randomization or augmentation.

It differs from ordinary image augmentation because the perturbation is part of the generated gameplay environment, not an external transformation applied after recording. Nevertheless, its training effect is similar: it forces the model to rely on robust visual cues rather than brittle pixel-perfect appearances.

Let γ_i denote the Ghost Mode condition for section i . Then the observation process can be represented as

$$I_t = \Omega_{\gamma_i}(s_t), \quad o_t = P_{\text{pre}}(I_t).$$

The model should be evaluated both with and without strong Ghost Mode conditions to determine whether this visual randomization improves robustness or harms precision.

5 Model Stages and Learning Objectives

5.1 Stage 1: Reactive Behavioral Cloning

Stage 1 trains a visual policy through behavioral cloning. It is both a baseline and a pretraining stage.

The input is a stack of preprocessed observations:

$$x_t = o_{t-k+1:t}.$$

A visual encoder maps the stack into an embedding:

$$z_t = E_\theta(x_t).$$

A binary prediction head outputs a probability:

$$p_t^{\text{react}} = \sigma(Wz_t + b),$$

where

$$p_t^{\text{react}} = \pi_\theta(a_t = 1 \mid x_t).$$

The model is trained using behavioral cloning:

$$\theta^* = \arg \min_{\theta} \mathcal{L}_{\text{BC}}(\theta).$$

The preferred first implementation is:

CNN encoder $\rightarrow$ MLP binary head.

A CNN should be preferred over a Transformer in the first version because it is simpler, faster, and likely more sample-efficient for this visual-control baseline. A Transformer or ViT can be introduced later as an ablation or higher-capacity model.

Stage 1 should be evaluated offline and online. Offline accuracy alone is insufficient. A model can achieve high frame-level accuracy but fail catastrophically if it makes rare timing errors at critical points. Therefore, Stage 1 must also be deployed in real gameplay to measure survival and progress.

5.2 Why Stage 1 Is Not Enough

Stage 1 is intentionally limited. Even with a frame stack, it is not a full history-based policy. It may fail in situations where the correct action depends on hidden or accumulated state.

Examples include:

- current vertical velocity,
- gravity direction,
- previous held duration,
- whether a jump has already been committed,
- section context under Ghost Mode,
- future trajectory after an orb or pad,
- mode transition immediately after a portal,
- recovery from slight timing deviations.

This motivates Stage 2.

5.3 Stage 2: Memory-Based Sequential Policy

Stage 2 trains a memory-based policy. The preferred first serious memory architecture is a GRU. The Stage 1 visual encoder is reused on the same frame-stack input:

$$z_t = E_\theta(x_t).$$

The recurrent model receives the visual embedding and the previous action:

$$u_t = [z_t, a_{t-1}].$$

The hidden state is updated by:

$$h_t = \text{GRU}_\phi(h_{t-1}, u_t).$$

The action probability is:

$$p_t^{\text{seq}} = \sigma(Wh_t + b).$$

Thus:

$$\pi_{\theta, \phi}(a_t = 1 \mid o_{\leq t}, a_{< t}) = p_t^{\text{seq}}.$$

The previous action is included because Geometry Dash dynamics are input-history-dependent. Whether the button was held in the previous frame affects current velocity and future trajectory.

The recommended architecture progression is:

$$\text{CNN} + \text{frame stack} \rightarrow \text{CNN} + \text{GRU} \rightarrow \text{CNN} + \text{LSTM} \rightarrow \text{CNN/ViT} + \text{Transformer}.$$

Stage 2 should initialize from Stage 1. This means the visual encoder learned during reactive behavioral cloning is reused, then fine-tuned with the recurrent model.

Two training modes should be considered:

1. freeze E_θ initially and train only the recurrent head,
2. unfreeze and fine-tune the full network end-to-end.

The sequence length L is a hyperparameter:

$$L \in \{8, 16, 32, 64, 128, \dots\}.$$

Shorter sequences are easier to train but may miss long-range context. Longer sequences may improve memory but increase compute and overfitting risk.

5.4 Sequential Behavioral Cloning Objective

For a trajectory

$$\tau = ((o_1, a_1), \dots, (o_T, a_T)),$$

Stage 2 minimizes:

$$\mathcal{L}_{\text{SBC}}(\theta, \phi) = - \sum_{t=1}^T [w_1 a_t \log p_t + w_0 (1 - a_t) \log(1 - p_t)].$$

Here:

$$p_t = \pi_{\theta, \phi}(a_t = 1 \mid o_{\leq t}, a_{< t}).$$

Training should use truncated backpropagation through time. A sequence batch contains contiguous subsequences rather than randomly shuffled frames. This preserves temporal structure.

Let a training window be:

$$W_i = ((o_i, a_i), \dots, (o_{i+L-1}, a_{i+L-1})).$$

The recurrent state may be reset at the start of each window or warmed up using preceding observations. Warm-up is preferable if compute allows it.

5.5 Exposure Bias and Compounding Error

Behavioral cloning trains the policy on histories visited by the human demonstrator. At deployment, the agent may visit histories caused by its own small mistakes. These histories may be outside the training distribution. A small error can therefore compound into a death.

This is the classic exposure-bias problem of imitation learning.

Let $d_{\mathcal{H}}^{\pi^H}$ be the history visitation distribution induced by the human policy and let $d_{\mathcal{H}}^{\pi}$ be the history visitation distribution induced by Voxter. Behavioral cloning trains on:

$$\mathcal{H}_t \sim d_{\mathcal{H}}^{\pi^H}.$$

Deployment occurs under:

$$\mathcal{H}_t \sim d_{\mathcal{H}}^{\pi}.$$

If:

$$d_{\mathcal{H}}^{\pi} \neq d_{\mathcal{H}}^{\pi^H},$$

then high offline imitation accuracy does not guarantee online success.

This is why the project should include reinforcement learning in the first full version. RL can fine-tune the policy on histories induced by the agent itself.

5.6 Stage 3: Reinforcement Fine-Tuning

Stage 3 uses reinforcement learning to improve the imitation-initialized policy through real interaction with the environment.

The policy begins from Stage 2 parameters:

$$\pi_0 = \pi_{\text{BC+memory}}.$$

The RL objective is to maximize expected return:

$$J(\pi) = \mathbb{E}_{\tau \sim p_{\pi}(\tau)} \left[\sum_{t=0}^T \gamma^t r_t \right],$$

where $p_{\pi}(\tau)$ is the closed-loop trajectory distribution induced by the policy, environment dynamics, initial conditions, generator distribution, and termination rule. The term r_t is the reward and γ is a discount factor.

When the implementation is expressed as a minimization problem, $\mathcal{L}_{\text{RL}}$ may denote either the negative-return loss

$$\mathcal{L}_{\text{RL}} = -J(\pi)$$

or an algorithm-specific policy-gradient surrogate. The chosen RL algorithm must define this sign convention explicitly.

The reward must encourage real gameplay progress without creating degenerate behavior. A first reward design is:

$$r_t = \alpha \Delta \text{progress}_t + \beta \text{alive}_t + \zeta \text{section}_t - \lambda \text{death}_t - \mu \text{instability}_t.$$

The terms are:

- $\Delta \text{progress}_t$: forward progress since the previous tick,
- alive_t : small survival reward,
- section_t : reward for passing a section or checkpoint-like boundary,
- death_t : penalty for dying,
- instability_t : optional penalty for unnecessary input flickering.

The weights

$$\alpha, \beta, \zeta, \lambda, \mu$$

must be tuned carefully.

The RL stage should not erase the imitation prior. It should refine it. Possible regularization:

$$\mathcal{L}_{\text{RL+BC}} = \mathcal{L}_{\text{RL}} + \kappa \mathcal{L}_{\text{BC}},$$

or a KL penalty to keep the fine-tuned policy near the imitation policy:

$$\mathcal{L} = \mathcal{L}_{\text{RL}} + \kappa \mathbb{E}_{\mathcal{H}_t \sim d_{\mathcal{H}}^{\pi}} [D_{\text{KL}}(\pi_{\text{RL}}(\cdot \mid \mathcal{H}_t) \parallel \pi_{\text{BC}}(\cdot \mid \mathcal{H}_t))].$$

The purpose is to improve robustness without destroying learned human-like gameplay structure.

6 Evaluation, Deployment, and Runtime Contracts

6.1 Real-Time Deployment Contract

Voxter must play in real time exactly like a human. It must not pause the game, slow the game, inspect future frames, read privileged game memory for control, or use level-specific scripts.

At deployment, each control cycle must satisfy:

$$T_{\text{capture}} + T_{\text{preprocess}} + T_{\text{inference}} + T_{\text{input}} \leq T_{\text{tick}},$$

where T_{tick} is the available time per game tick or deployed sampling interval.

For 60 Hz control:

$$T_{\text{tick}} \approx 16.67 \text{ ms.}$$

For 120 Hz control:

$$T_{\text{tick}} \approx 8.33 \text{ ms.}$$

The runtime system must define:

1. frame capture method,

2. preprocessing pipeline,
3. model inference schedule,
4. input execution method,
5. action smoothing or thresholding,
6. dropped-frame behavior,
7. fail-safe behavior when inference misses a deadline.

If the model outputs a probability p_t , deployment uses a threshold:

$$a_t = \mathbf{1}\{p_t > \eta\}.$$

The threshold η is tunable. It may be globally fixed or calibrated per model. A hysteresis mechanism may reduce flickering:

$$\eta_{\text{press}} > \eta_{\text{release}}.$$

For example:

$$\begin{aligned} a_t &= 1 && \text{if } p_t > \eta_{\text{press}}, \\ a_t &= 0 && \text{if } p_t < \eta_{\text{release}}, \end{aligned}$$

and otherwise retain the previous action.

This should be evaluated carefully because excessive smoothing can make the agent late.

6.2 Offline Evaluation Protocol

Offline evaluation measures how well the model imitates recorded demonstrations. It is necessary but not sufficient.

The recommended offline metrics are:

1. binary cross-entropy,
2. accuracy,
3. balanced accuracy,
4. precision, recall, and F1 for held input,
5. transition precision and recall,
6. press timing error,
7. release timing error,
8. per-mode performance,
9. per-Ghost-Mode performance,
10. per-section or held-out-section performance.

Frame-level accuracy can be misleading. Suppose the correct action is released for 90% of frames. A model that always releases gets 90% accuracy but cannot play. Therefore balanced metrics and transition metrics are essential.

Transition metrics are especially important because deaths often occur from incorrect press or release timing rather than from long-duration state mismatch.

Let predicted action be $\hat{a}_t$. A predicted press event is:

$$\hat{e}_t^{\text{press}} = \mathbf{1}\{\hat{a}_{t-1} = 0 \wedge \hat{a}_t = 1\}.$$

A predicted release event is:

$$\hat{e}_t^{\text{release}} = \mathbf{1}\{\hat{a}_{t-1} = 1 \wedge \hat{a}_t = 0\}.$$

Timing error can be measured by matching predicted transitions to demonstration transitions within a tolerance window.

Let

$$\hat{\mathcal{T}}_{\text{press}} = \{t : \hat{e}_t^{\text{press}} = 1\}, \quad \mathcal{T}_{\text{press}} = \{t : e_t^{\text{press}} = 1\}.$$

For a demonstration press at time $t \in \mathcal{T}_{\text{press}}$, define the nearest predicted press timing error as

$$\text{err}_{\text{press}}(t) = \min_{\hat{t} \in \hat{\mathcal{T}}_{\text{press}}} |\hat{t} - t|.$$

A press is counted as matched under tolerance Δ when $\text{err}_{\text{press}}(t) \leq \Delta$. Release timing error is defined analogously using $\hat{\mathcal{T}}_{\text{release}}$ and $\mathcal{T}_{\text{release}}$.

6.3 Online Evaluation Protocol

Online evaluation is the true test of the model. The agent must play the game under the same real-time conditions expected during deployment.

The recommended online metrics are:

1. survival time,
2. progress percentage,
3. section completion rate,
4. full-level completion rate,
5. death location distribution,
6. mean attempts to completion,
7. held-out-section success rate,
8. standard-level transfer performance,
9. performance under Ghost Mode,
10. latency and inference deadline misses.

Online evaluation should be performed at three levels.

6.3.1 Training-Generator Evaluation

This tests performance on the same distribution used for training. It helps detect whether the model can play at all.

6.3.2 Held-Out-Generator Evaluation

This tests performance on sections removed during training and added back during testing. It measures in-generator generalization.

6.3.3 Standard-Level Evaluation

This tests performance on normal levels outside the generator. It measures out-of-generator transfer, the most ambitious goal.

The final success criterion is strong standard-level performance:

A trained Voxter agent should be able to complete entire standard levels easily, under real-time conditions, without timestamp scripts.

6.4 Baselines

Even if time is limited, baselines are important for interpreting results.

The most important baselines are:

1. Timestamp bot: executes a fixed click script. Expected to fail on unseen levels.
2. Always release: measures the difficulty of action imbalance.
3. Always hold: another trivial action baseline.
4. Reactive CNN baseline: Stage 1 model.
5. CNN + frame stack: strengthened Stage 1 baseline.
6. CNN with GRU memory: first memory baseline.
7. CNN with LSTM memory: recurrent comparison.
8. Transformer temporal model: high-capacity temporal comparison, if resources allow.
9. Human demonstrator: upper reference for the data collection distribution.

The timestamp bot is philosophically central. Voxter should be evaluated against it because the main claim is not that it can replay a known level, but that it can perform better than timestamp replay on unseen levels.

6.5 Model Selection and Hyperparameters

Important tunable parameters include:

- model input resolution (H, W) ,
- frame-stack length k ,
- recurrent sequence length L ,
- CNN encoder depth,
- GRU hidden size,

- action threshold η ,
- hysteresis thresholds $\eta_{\text{press}}, \eta_{\text{release}}$,
- class weights w_0, w_1 ,
- learning rate,
- batch size,
- RL reward weights,
- BC/RL regularization strength,
- label alignment offset δ .

The most dangerous hyperparameters are those that affect timing:

$$k, L, \delta, \eta, \eta_{\text{press}}, \eta_{\text{release}}.$$

A model can appear good offline but fail online if these are poorly calibrated.

6.6 Data Splitting Rules

Data should not be split randomly by frame. Random frame splitting leaks section and trajectory information between train and test.

Bad split:

$$\text{random frames} \rightarrow \text{train/test}.$$

Better split:

$$\text{trajectories, sections, seeds, or section families} \rightarrow \text{train/test}.$$

The strongest split is section-level:

$$Z_{\text{train}} \cap Z_{\text{test}} = \emptyset,$$

where Z_{train} and Z_{test} are sets of section configurations or section identities.

Since the testing set includes brand new sections added back to the generator, the document defines the held-out generator test as a core benchmark.

6.7 Partial Observability and Memory Justification

The game is partially observable from pixels because the rendered frame does not fully expose all state variables. Even when the current frame is visually clear, the correct action may depend on dynamic variables such as velocity.

A recurrent policy maintains an internal state:

$$h_t = R(h_{t-1}, o_t, a_{t-1}).$$

This hidden state approximates sufficient information from the past:

$$h_t \approx \varphi(o_{\leq t}, a_{< t}).$$

The policy then acts as:

$$\pi(a_t \mid h_t).$$

This is why a memory architecture is not merely an implementation choice. It is theoretically justified by the partial observability of the control problem.

7 Implementation Interface, Risks, and Plan

7.1 Implementation Architecture

The deployed system can be described as a loop:

1. capture frame,
2. preprocess frame,
3. update observation stack or recurrent state,
4. run model inference,
5. threshold output probability,
6. send held/released input state,
7. log runtime data,
8. repeat at the target rate.

In pseudocode:

```
initialize recurrent state h_0
initialize previous action a_0 = 0

for each control tick t:
    I_t = capture_frame()
    o_t = preprocess(I_t)
    z_t = encoder(o_t)
    h_t = memory(h_{t-1}, concat(z_t, a_{t-1}))
    p_t = sigmoid(head(h_t))
    a_t = threshold_or_hysteresis(p_t, a_{t-1})
    apply_input_state(a_t)
    log(I_t, o_t, p_t, a_t, timestamp_t)
```

The exact runtime implementation may differ, but it must preserve the same information constraints.

7.2 Reinforcement Learning Environment Interface

For RL, the game must expose an environment-like interface:

$$(o_t, r_t, done_t, info_t) = \text{Env.step}(a_t).$$

However, because the actual game is real-time and not a standard simulator, the interface may be implemented as a wrapper around capture and input control.

The environment must provide:

- observations from screen capture,
- actions as held/released input states,

- rewards based on progress and survival,
- terminal signal on death or completion,
- reset mechanism after death or level completion,
- logging for analysis.

If precise progress cannot be read directly, progress may be estimated through visual cues, elapsed time, or level position if accessible for evaluation. Any metadata used only for reward or evaluation must be carefully separated from what the policy observes.

7.3 Privileged Information Rule

Voxter may use metadata for training diagnostics, reward computation, evaluation, or dataset balancing. But the deployed policy should not receive privileged information that a human player would not receive visually.

Allowed for model input:

- captured frames,
- preprocessed frames,
- previous actions,
- recurrent memory computed from past observations.

Allowed for logging/evaluation/reward, but not necessarily for policy input:

- section ID,
- random seed,
- speed metadata,
- mode metadata,
- death state,
- progress,
- completion state.

Disallowed for policy input in the main real-time claim:

- future frames,
- exact future obstacle positions not visible on screen,
- hand-coded click timestamps,
- complete level script,
- symbolic level representation unavailable to a human player.

7.4 Failure Modes and Limitations

The project should explicitly acknowledge likely failure modes.

7.4.1 Covariate Shift

The model may enter states that the human demonstrator rarely visited. This can lead to compounding errors.

7.4.2 Latency Misalignment

A small label offset can make the model consistently early or late. Geometry Dash is highly sensitive to timing, so causal alignment is critical.

7.4.3 Action Imbalance

If released frames dominate the dataset, the model may learn a passive policy.

7.4.4 Overfitting to Generator Style

The model may learn the visual style of RobTop-derived training sections without fully generalizing to arbitrary user-created levels.

7.4.5 Ghost Mode Ambiguity

Faded obstacles may make some situations visually ambiguous. Domain randomization helps robustness but may also reduce precision if too aggressive.

7.4.6 Behavioral Cloning Brittleness

Behavioral cloning does not teach recovery from the agent’s own mistakes unless those states appear in the demonstrations.

7.4.7 Real-Time Compute Limits

A model that is accurate but too slow is invalid for the real-time claim.

7.4.8 Reward Hacking

The RL stage may exploit poorly designed rewards. Reward terms must be tested for degenerate behavior.

7.4.9 Standard-Level Transfer Gap

Completing standard levels may require more than general visual affordances. Some levels depend on memorization, deceptive design, invisible triggers, or nonstandard gameplay conventions.

7.5 Ethical and Project-Scope Note

Voxter is a research and engineering project about visual control, imitation learning, reinforcement learning, and real-time gameplay agents. The intended use is experimentation, understanding, and project development. Public release should clearly communicate limitations, avoid misrepresenting performance, and distinguish research demonstrations from competitive or leaderboard use.

7.6 First-Version Development Plan

The recommended first-version sequence is:

1. implement reliable capture and input logging,
2. measure synchronization and latency,
3. build preprocessing pipeline,
4. create dataset format,
5. train Stage 1 CNN + frame-stack behavioral cloning,
6. deploy Stage 1 online for sanity testing,
7. train Stage 2 CNN + GRU sequential policy,
8. evaluate on held-out generator sections,
9. implement RL wrapper and reward system,
10. fine-tune Stage 2 policy with RL,
11. evaluate on standard levels,
12. perform ablations and document results.

The first working model should prioritize correctness and measurable learning over architectural complexity.

7.7 Minimal Dataset Schema

A practical dataset record may use the following conceptual schema:

Field	Meaning
<code>sample_id</code>	unique sample identifier
<code>run_id</code>	run identifier
<code>frame_index</code>	frame index inside the run
<code>timestamp</code>	capture timestamp or game-tick index
<code>raw_frame_path</code>	raw frame path
<code>preprocessed_frame_path</code>	preprocessed frame path
<code>action_held</code>	binary held-state action
<code>alive</code>	whether the attempt is still active
<code>done</code>	terminal flag indicating death, completion, or manual reset
<code>attempt_id</code>	attempt identifier
<code>section_id_optional</code>	optional generated-section identifier
<code>mode_optional</code>	optional mode metadata
<code>speed_optional</code>	optional speed metadata
<code>ghost_mode_optional</code>	optional Ghost Mode metadata
<code>progress_optional</code>	optional progress metadata
<code>notes_optional</code>	optional notes

For sequence training, records should be grouped by run and ordered by frame index. Shuffling should occur at the sequence-window level, not individual-frame level.

7.8 Open Questions for Later Iteration

Several questions remain for later empirical resolution:

1. best input resolution,
2. best frame-stack length,
3. whether icon standardization helps or harms,
4. whether grayscale loses important information,
5. best recurrent sequence length,
6. best action thresholding strategy,
7. whether GRU is sufficient or LSTM/Transformer improves performance,
8. best RL algorithm for this real-time wrapper,
9. best progress signal for reward computation,
10. best balance between imitation regularization and RL exploration.

These are not blockers for the first draft. They are experimental variables.

7.9 Conclusion

Voxter should be understood as a real-time visuomotor policy agent, not a timestamp bot. The training level is a randomized generator of gameplay situations, not the final task. Human demonstrations provide the initial behavioral structure. A memory-based policy captures the partial observability of the game. Reinforcement learning then fine-tunes the agent on its own interaction distribution.

The theory of Voxter rests on four principles:

1. Visual causality: actions must be predicted from information available at decision time.
2. Binary held-state control: the correct action representation is held/released state, not merely click events.
3. Domain-randomized training: randomized sections force the model away from timestamp memorization.
4. Real-time deployment: the final agent must act under the same temporal constraints as a human player.

If these principles are preserved, Voxter can be developed as a serious attempt to learn transferable Geometry Dash gameplay behavior from screen input.

A Notation Table

Symbol	Meaning
s_t	unobserved environment state at time t
I_t	raw captured frame with shape $H_0 \times W_0 \times 3$
o_t	preprocessed visual observation
x_t	short frame stack used by the reactive policy
a_t	held/released input state applied over $[t, t + 1)$
$\mathcal{H}_t$	observation-action history available at decision time
G	randomized gameplay generator
P_G	section-configuration distribution induced by G
P_{G, π_H}	trajectory distribution induced by G and human policy π_H
$d_{\mathcal{H}}^{\pi}$	history visitation distribution induced by policy π
δ_{sys}	system-delay offset; positive means action logs lag visual frames
p_t^{react}	reactive held-input probability $\pi_{\theta}(a_t = 1 \mid x_t)$
p_t^{seq}	sequential held-input probability $\pi_{\theta, \phi}(a_t = 1 \mid o_{\leq t}, a_{< t})$

B Compact Mathematical Summary

$I_t \in \mathbb{R}^{H_0 \times W_0 \times 3}$	raw frame,
$o_t = P_{\text{pre}}(I_t)$	preprocessed observation,
$x_t = o_{t-k+1:t}$	frame stack,
$a_t \in \{0, 1\}$	binary held-state action,
$e_t^{\text{press}} = \mathbf{1}\{a_{t-1} = 0 \wedge a_t = 1\}$	press event,
$e_t^{\text{release}} = \mathbf{1}\{a_{t-1} = 1 \wedge a_t = 0\}$	release event,
$p_t^{\text{react}} = \pi_{\theta}(a_t = 1 \mid x_t)$	reactive policy,
$z_t = E_{\theta}(x_t)$	visual embedding,
$h_t = R_{\phi}(h_{t-1}, [z_t, a_{t-1}])$	memory update,
$p_t^{\text{seq}} = \pi_{\theta, \phi}(a_t = 1 \mid o_{\leq t}, a_{< t})$	sequential policy.

The behavioral cloning loss is:

$$\mathcal{L}_{\text{BC}} = - \sum_t [w_1 a_t \log p_t + w_0 (1 - a_t) \log(1 - p_t)].$$

The RL objective is:

$$J(\pi) = \mathbb{E}_{\tau \sim p_{\pi}(\tau)} \left[\sum_{t=0}^T \gamma^t r_t \right].$$

The real-time constraint is:

$$T_{\text{capture}} + T_{\text{preprocess}} + T_{\text{inference}} + T_{\text{input}} \leq T_{\text{tick}}.$$

C Project Source Notes

This first draft is based on the project planning answers contained in `answers.odt` and on the training-level description contained in `TrainingLevelDescription.txt`.

The training-level source describes the level as an almost totally random endless run of random sections from main RobTop levels, where new sections may change the player form, cosmetic icon appearance, gameplay speed, Ghost Mode fading, and randomized paths.

The planning source specifies the desired document style, target audience, central generalization claim, action definition, observation strategy, training stages, evaluation ambition, real-time constraint, and intention to include reinforcement learning in the first version.